

Bus Ticket Booking System

Project Report

Course: Advanced Programming Applications (CS244)

Submitted to: ENG Shadi

Date: May 2026

Team Members:

- Abdulrahman Keshk
- Ahmad mahgoub

Table of Contents

1. Introduction
2. Project Objectives
3. System Architecture
4. Network Protocol Design
5. Class Diagram & OOP Concepts
6. Implementation Highlights
7. User Interface Screens
8. Challenges & Solutions
9. Team Contribution
10. Conclusion & Future Work

1. Introduction

The Bus Ticket Booking System is a client-server desktop application developed in Java that allows users to register, login, browse available bus trips between Egyptian cities, book seats in real-time, and view their bookings.

The system demonstrates advanced programming concepts including OOP principles, TCP socket networking, multithreading, JavaFX GUI, and proper exception handling.

2. Project Objectives

- Implement a real client-server architecture using TCP sockets.
- Apply OOP concepts (Encapsulation, Inheritance, Polymorphism, Abstraction).
- Ensure thread-safety for concurrent booking.
- Provide a user-friendly JavaFX interface.
- Handle network communication reliably with a custom protocol.
- Demonstrate practical use of multithreading on the server side.

3. System Architecture

The system follows a layered client-server architecture:

- Model Layer: Contains business entities (User, Passenger, Trip, Booking).
- Data Layer: DataManager (Singleton pattern) manages in-memory data.
- Network Layer: Server + ClientHandler using thread pool.
- Client Layer: JavaFX controllers + ClientNetwork utility.

Communication: Pure TCP sockets with a simple text-based protocol.

4. Network Protocol

Command	Request Example	Response Example
REGISTER	REGISTER username pass Name Phone	REGISTER_SUCCESS
LOGIN	LOGIN username pass Ahmed_Essam	LOGIN_SUCCESS
GET_TRIPS	GET_TRIPS T001 Cairo Alex...#T002 ...	TRIPS
BOOK	BOOK T001 A5 B1746123456789	BOOK_SUCCESS
MY_BOOKINGS	MY_BOOKINGS B001 T001 A5 250#...	MYBOOKINGS

5. Class Diagram & OOP Concepts

Main Classes (More than 8 classes):

- User (Abstract)
- Passenger (Inheritance)
- Trip (Serializable + synchronized booking)
- Booking (Serializable)
- DataManager (Singleton)
- BookingException (Custom Exception)
- Server + ClientHandler (Multithreading)
- Multiple JavaFX Controllers

OOP Applied:

- Abstraction: Abstract User class
- Encapsulation: Private fields + getters/setters
- Inheritance: Passenger extends User
- Polymorphism: Overriding toString()
- Singleton: DataManager

6. Implementation Highlights

Server Side:

- Uses ExecutorService (Cached Thread Pool) for handling multiple clients.
- Thread-safe booking using synchronized methods.

Client Side:

- Clean JavaFX GUI with multiple scenes.
- ClientNetwork utility class for sending requests.

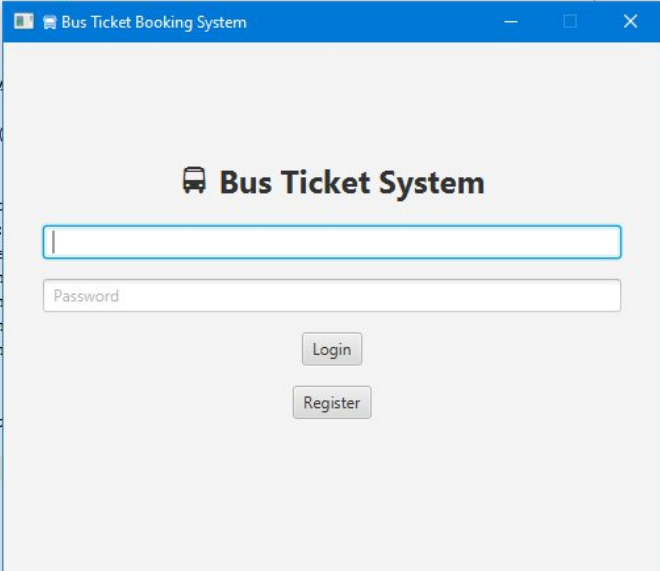
Key Features:

- Real-time seat availability
- Persistent login session per client
- Proper error handling across network

7. User Interface Screens

The application consists of the following screens:

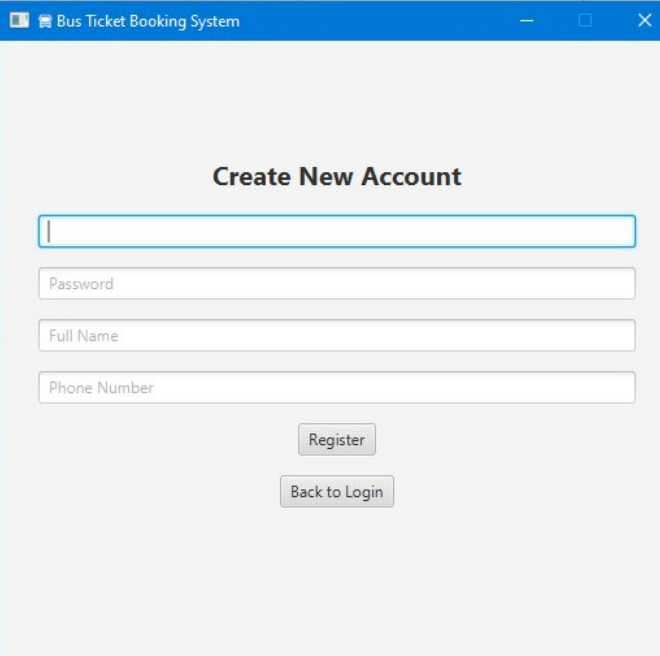
1. Login Screen



The screenshot shows a window titled "Bus Ticket Booking System". The main heading is "Bus Ticket System" with a bus icon. Below the heading are two input fields: the first is empty, and the second is labeled "Password". Below the input fields are two buttons: "Login" and "Register".

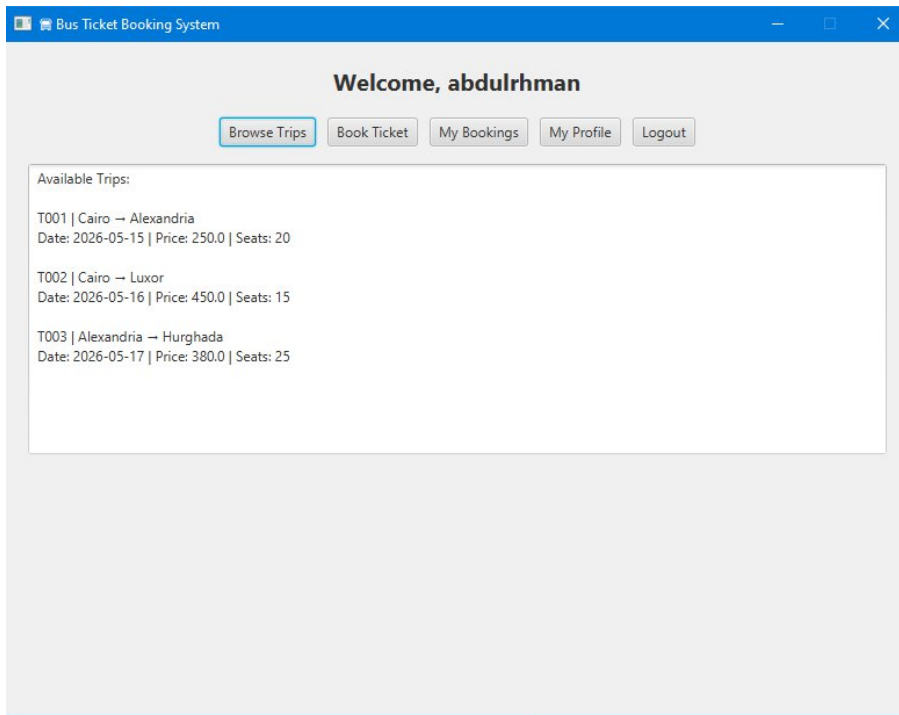
2. Registration Screen

```
("BOOK_FAILED " + e.getMessage());
```



The screenshot shows a window titled "Bus Ticket Booking System". The main heading is "Create New Account". Below the heading are four input fields: the first is empty, the second is labeled "Password", the third is labeled "Full Name", and the fourth is labeled "Phone Number". Below the input fields are two buttons: "Register" and "Back to Login".

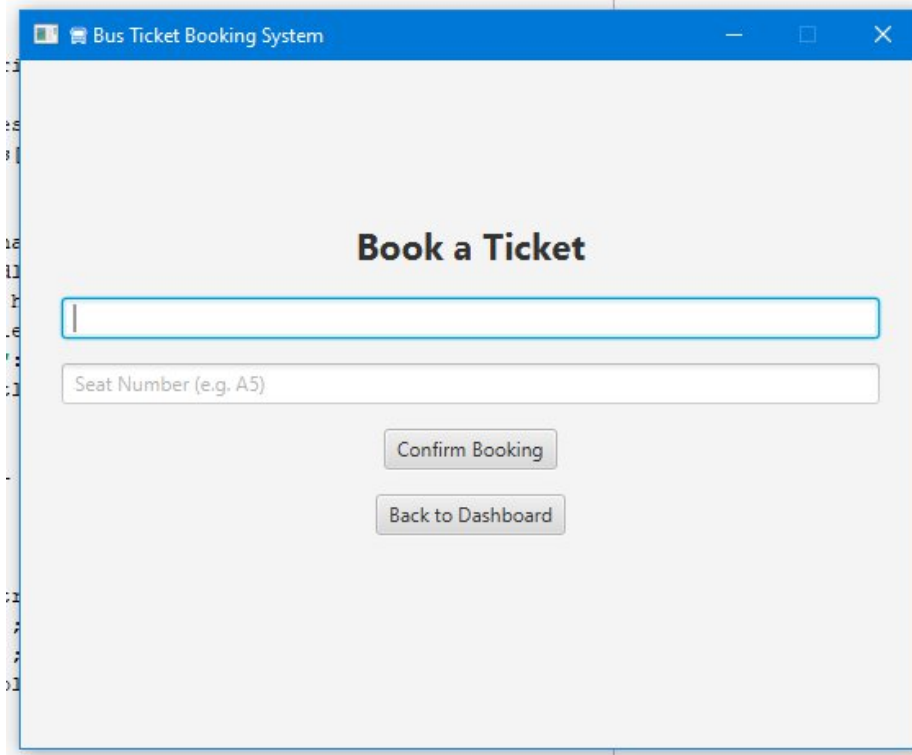
3. Dashboard (with Browse Trips)



The screenshot shows a web application window titled "Bus Ticket Booking System". The main heading is "Welcome, abdulrhman". Below this, there are five buttons: "Browse Trips" (highlighted with a blue border), "Book Ticket", "My Bookings", "My Profile", and "Logout". A section titled "Available Trips:" contains three entries:

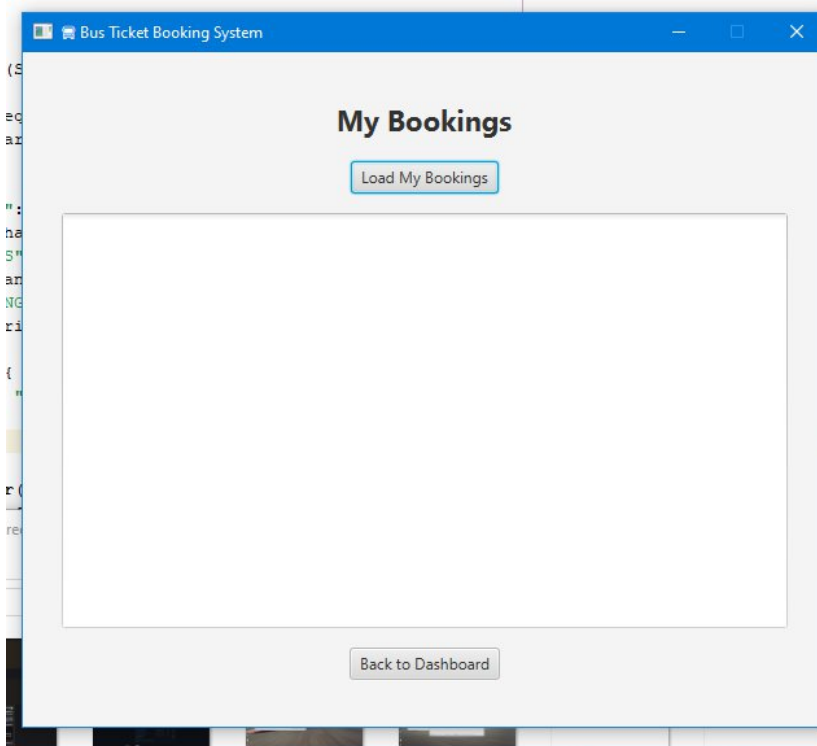
- T001 | Cairo → Alexandria
Date: 2026-05-15 | Price: 250.0 | Seats: 20
- T002 | Cairo → Luxor
Date: 2026-05-16 | Price: 450.0 | Seats: 15
- T003 | Alexandria → Hurghada
Date: 2026-05-17 | Price: 380.0 | Seats: 25

4. Book Ticket Screen

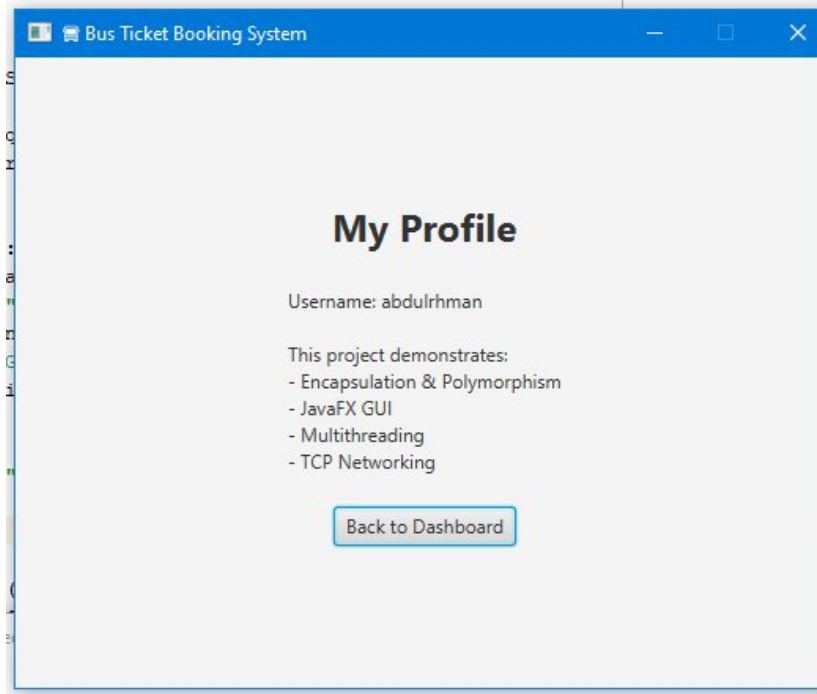


The screenshot shows a web application window titled "Bus Ticket Booking System". The main heading is "Book a Ticket". Below this, there are two input fields: a long text field and a shorter text field labeled "Seat Number (e.g. A5)". Below the input fields, there are two buttons: "Confirm Booking" and "Back to Dashboard".

5. My Bookings Screen



6. Profile Screen



8. Challenges & Solutions

Challenge	Solution Implemented
----- -----	
Race condition on seat booking	Added synchronized on bookSeat()
Network connection management	Reusable ClientNetwork class
String parsing complexity	Well-defined protocol with delimiters (#)
GUI organization	Separate Controller for each screen

9. Team Contribution

Abdulrahman Keshk:

- Server & Networking (Server.java, ClientHandler.java)
- All Model classes + DataManager
- Multithreading & Thread safety
- Exception handling

Ahmad mahgoub:

- All JavaFX GUI + Controllers
- ClientNetwork.java
- UI Design & User Experience
- Report documentation & README

10. Conclusion & Future Work

This project successfully demonstrates a full client-server application using core Java technologies. It meets all the advanced programming course requirements and provides a functional booking system.

Future Enhancements:

- Persistent storage using JSON or Database
- Admin panel for managing trips
- Real-time seat updates (broadcasting)
- Payment integration
- Using FXML + Scene Builder